

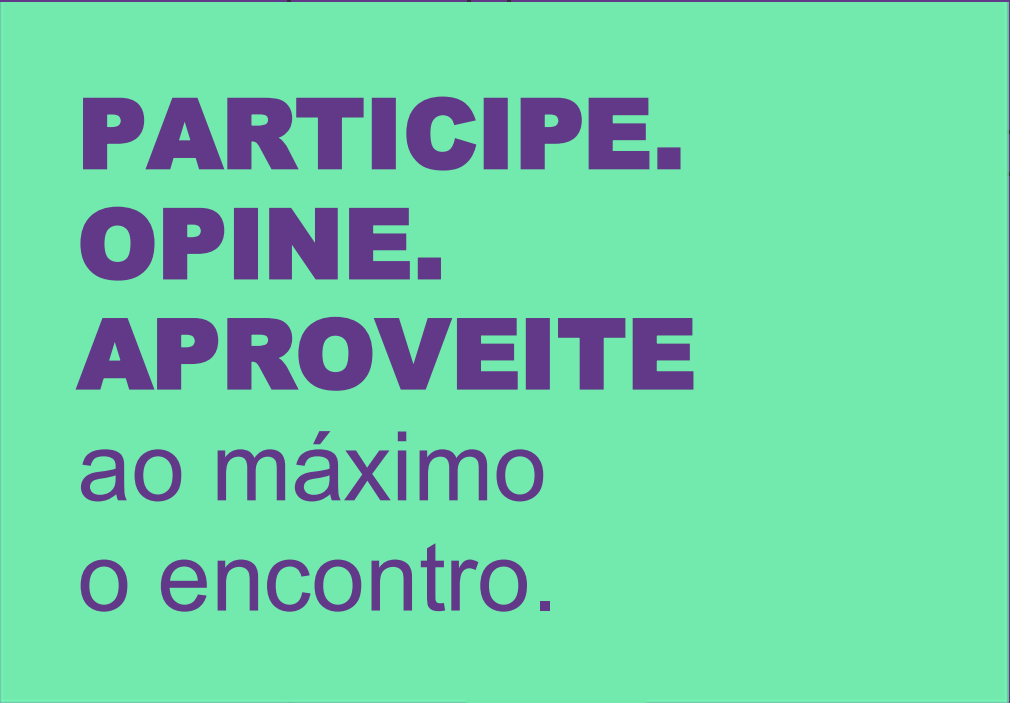
Programação em Python

Desenvolver Aplicações Back- End para Web com Python

Bem-vindos à Aula:
Desenvolver Aplicações
Back-End para web com
python

O que faremos hoje:

- Apresentação
- Revisão de Logica
- Basico de Github



**PARTICIPE.
OPINE.
APROVEITE**
ao máximo
o encontro.

Estamos em um ambiente
heterogêneo e diverso,
onde sempre há respeito.

Então, lembre-se de que:

**NÃO EXISTE
RESPOSTA ERRADA,
PERGUNTA BOBA**
ou que não valha a pena.

Para não termos interrupções e em respeito aos colegas,
pedimos que não faça uso de celular durante a aula e que
o coloque no modo silencioso.

Este curso tem como objetivo capacitar o aluno para desenvolver algoritmos utilizando **lógica de programação, estruturar aplicações front-end e desenvolver aplicações back-end para web com Python.**

Unidades Curriculares	Carga horária
UC1: Desenvolver algoritmos utilizando lógica de programação	48 horas
UC2: Desenvolver aplicações <i>front-end</i> para web	36 horas
UC3: Desenvolver aplicações <i>back-end</i> para web com Python	84 horas
Carga Horária Total	168 horas



Indicadores



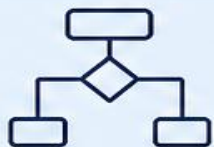
COMPETÊNCIAS

1



Configura o ambiente de desenvolvimento conforme as funcionalidades e características da aplicação web.

2



Estrutura algoritmos com base na lógica computacional e nos requisitos funcionais descritos na aplicação web.

3



Cria estruturas de código utilizando linguagem de programação para backend de acordo com os requisitos do projeto de software.

4



Desenvolve tratamento de requisições POST e GET do front-end de acordo com a linguagem de programação back-end.

5



Implementa session e cookies da aplicação web de acordo com a linguagem de programação back-end.

6



Integra banco de dados de acordo com a aplicação web e com os requisitos do projeto de software.



Avaliação



DURANTE O PROCESSO



A

Atendido

Mandou bem!



PA

**Parcialmente
Atendido**

Quase lá ...



NA

Não Atendido

Vamos tentar de novo ...

AO FINAL DO CURSO



A

Atendido

Concluiu com sucesso!



NA

Não Atendido

Ainda não deu ...



Situação de Aprendizagem





SITUAÇÃO DE APRENDIZAGEM



Desafio: Processo Seletivo – JWC Tecnologia



A JWC Tecnologia é uma empresa de desenvolvimento de aplicações web, que atende clientes de médio e grande porte.



Localização: Centro do Rio de Janeiro



Equipe: 30 colaboradores na área administrativa

- 20 desenvolvedores com diferentes perfis

Com o aumento da demanda – principalmente na área da educação – a empresa precisa **expandir sua equipe**.



Vaga Aberta: Programador Full Stack

O RH decidiu realizar um processo seletivo **por etapas**, com desafios técnicos progressivos,

- A cada desafio concluído, o candidato avança.



Pergunta Final:

E você? Está pronto para essa vaga?



Perfil desejado

- ✓ Curiosas
- ✓ Ágeis
- ✓ Inovadoras
- ✓ Apaixonadas por tecnologia
- ✓ Motivadas a enfrentar desafios
- ✓ Comprometidas com melhoria contínua





Revisão de lógica





VAMOS PENSAR!



1

Onde usamos lógica no dia a dia?



2

Jogos possuem lógica?



3

Aplicativos como iFood, Uber e PIX usam decisões e repetições?



LÓGICA É A BASE DE TUDO! ELA ESTÁ EM **TODO LUGAR!**



Vamos pro código?





ATIVIDADE – LOJA DE GAMES



Crie um sistema em **Python** para simular uma loja de jogos!

SISTEMA DEVE:



Mostrar os jogos disponíveis



Permitir compra



Somar valores



Mostrar total da compra



Verificar desconto

REGRAS

Se a compra passar
de **R\$200** →
desconto de **10%**!



TABELA DE JOGOS

JOGO	VALOR (R\$)
 Minecraft	80
 GTA V	120
 FIFA	90



ATIVIDADE DOS ALUNOS

O aluno deverá criar um sistema que:



Use listas



Use repetição
(laço)



Use decisão
(condicional)



Some
valores



Mostre o
resultado final



EXEMPLO DE SAÍDA

Jogos disponíveis:

0 - Minecraft R\$ 80
1 - GTA V R\$ 120
2 - FIFA R\$ 90

Escolha os jogos para comprar.

Total da compra: **R\$ 210,00**

Desconto (10%): **R\$ 21,00**

Total com desconto: **R\$ 189,00**



DICA: Use listas, laços (*for/while*), condicionais (*if/else*) e operações matemáticas!





AÇÃO 2 – ARENA DE RPG

Criar um sistema que simule batalhas contra monstros!



SITUAÇÃO

Um jogador enfrentará 3 monstros.

Cada vitória:

perde vida

ganha moedas

100

0

O sistema deve:

- controlar vida
- controlar moedas
- mostrar status após cada batalha

REGRAS

MONSTRO	DANO	RECOMPENSA
Slime	10	20
Goblin	20	40
Dragão	50	100

DESAFIO DOS ALUNOS

Utilizar:

- Variáveis
- Listas
- Estrutura de repetição (for)
- Estrutura de decisão (if/else)

OBJETIVO:
Sobreviver aos 3 monstros e ganhar o máximo de moedas!

EXEMPLO DE FUNCIONAMENTO

1

Enfrentou: Slime

-10 de vida +20 moedas

Vida: 90 Moedas: 20

2

Enfrentou: Goblin

-20 de vida +40 moedas

Vida: 70 Moedas: 60

3

Enfrentou: Dragão

-50 de vida +100 moedas

Vida: 70 Moedas: 160

Você venceu a arena! Total de moedas: 160

DICA DE LÓGICA

for

Use o *for* para percorrer os monstros

Subtraia o dano da vida

Some a recompensa nas moedas

Use *if* para verificar se o jogador morreu

EXEMPLO DE SAÍDA (NO TERMINAL)

```
=== ARENA RPG ===
Sua vida inicial: 100
Suas moedas: 0

Você enfrentou: Slime
Você perdeu 10 de vida
Você ganhou 20 moedas
Vida atual: 90 | Moedas: 20

Você enfrentou: Goblin
Você perdeu 20 de vida
Você ganhou 40 moedas
Vida atual: 70 | Moedas: 60

Você enfrentou: Dragão
Você perdeu 50 de vida
Você ganhou 100 moedas
Vida atual: 20 | Moedas: 160

Você venceu a arena!
Total de moedas: 160
```

O QUE VOCÊ VAI PRATICAR?

Variáveis

Listas

Repetição

Decisão

Operações

Lógica

PENSE, PLANEJE, CODIFIQUE E VENÇA A ARENA!



AÇÃO 3 – CONTROLE DE ESTACIONAMENTO

Crie um sistema que **cadastre** carros, **mostre** vagas e permita **buscar** placas!



SITUAÇÃO

Um estacionamento tem **10 vagas**.

Precisamos de um sistema para:

- ✓ Cadastrar carros (placa)
- ✓ Mostrar carros estacionados
- ✓ Mostrar vagas ocupadas e vagas livres
- ✓ Buscar uma placa
- ✓ Permitir sair do sistema



FLUXO DE EXEMPLO

1



Carro entra
Digite a placa.

2



Sistema armazena
a placa na lista.

3



Sistema calcula
vagas ocupadas
e livres.

4



Usuário pode
buscar uma
placa.

5



Usuário pode
sair do sistema.



FUNCIONALIDADES DO SISTEMA

1



Adicionar carro

Digite a placa do carro que entrou.

2



Mostrar carros

Lista todos os carros estacionados,
vagas ocupadas e vagas livres.

3



Buscar placa

Digite a placa para verificar
se está estacionada.

4



Sair

Encerrar o sistema.



REGRAS

Capacidade do estacionamento:

10 vagas

Não é permitido:

- ✗ Adicionar mais carros que a capacidade.
- ✗ Cadastrar placas repetidas.

VAGAS OCUPADAS



6

VAGAS LIVRES



4



LÓGICA QUE VOCÊ VAI USAR



LISTAS

Para armazenar as placas dos carros.



REPETIÇÃO

Para mostrar o menu e listar carros.



DECISÃO

Para escolher o que fazer.



SOMA E CONTROLE

Para contar vagas ocupadas e livres.

EXEMPLO DE EXECUÇÃO (NO TERMINAL)

=== CONTROLE DE ESTACIONAMENTO ===

- 1 - Adicionar carro
- 2 - Mostrar carros
- 3 - Buscar placa
- 4 - Sair

Escolha uma opção: 1

Digite a placa: ABC-1234

Carro cadastrado com sucesso!

Escolha uma opção: 2

=== CARROS ESTACIONADOS ===

ABC-1234

DEF-5678

GHI-9101

Vagas ocupadas: 3

Vagas livres: 7

Escolha uma opção: 3

Digite a placa para buscar: DEF-5678

Carro encontrado!



O QUE VOCÊ VAI PRATICAR?



Listas



Repetição
(laços)



Decisão
(if/else)



Contagem e
cálculos



Busca em
lista



Interatividade
com o usuário



DESAFIO EXTRA

Não permitir placas repetidas e avisar se o estacionamento estiver cheio!



ATÉ A
PRÓXIMA
AULA!